Software Requirements Specification (SRS)

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Course: COEN838 — Advanced Software Engineering
Department: Computer Engineering, Ahmadu Bello University, Zaria
Student: Idris Muhammad Abubakar
Matric No: P25EGCP9003
Version: 1.1
Date: October 2026

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **ABU-SSO**, a Unified Single Sign-On and Digital Campus Wallet system for Ahmadu Bello University. The document serves as a blueprint for developers, testers, and stakeholders by specifying what the system must do, how well it must perform, and under what constraints it must operate. It is intended to bridge user needs (student frustration with multiple credentials) to precise technical requirements that the engineering team can implement and verify.

1.2 Scope

In scope:

- Central authentication service issuing JSON Web Tokens (JWT) for all ABU campus applications.
- **Matric / staff number-based login** — students and staff use their official university number as their identifier.
- **Admin-controlled account creation** — after the first (bootstrap) admin, no public self-registration is permitted.
- Digital ID card generation with QR code for physical access points (library, hostel).
- Campus wallet allowing top-up, payments, and transaction history.
- A web-based dashboard acting as the launchpad for connected campus services.
- **Six distributed Single Sign-On services** — Student Portal, LMS, Library, Hostel, Wi-Fi, Result Checker — each a separate application that independently verifies the ABU-SSO JWT.
- Role-Based Access Control (student / staff / admin) enforced by middleware.

Out of scope (for this prototype):

- Native mobile apps (iOS/Android).
- Integration with real university ERP (student portal, Moodle).
- Real payment gateways (top-ups are simulated).
- Biometric fingerprint/face authentication.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
SSO	Single Sign-On — one login grants access to multiple independent services
JWT	JSON Web Token — a signed, stateless authentication token
bcrypt	A password hashing algorithm designed for resistance to brute-force attacks
RBAC	Role-Based Access Control
IdP	Identity Provider — the service that authenticates users and issues tokens
Matric	Matriculation number — the unique identifier issued to every university student
Bootstrap	The first-ever user registration on a fresh deployment, automatically granted admin role
SRS	Software Requirements Specification
NFR	Non-Functional Requirement
FR	Functional Requirement
RTM	Requirements Traceability Matrix

1.4 References

- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill.
- FastAPI documentation — https://fastapi.tiangolo.com
- RFC 7519 (JWT), RFC 7518 (JWA)
- ABU ICT Directorate (internal stakeholder interviews).

1.5 Overview of This Document

- **Section 2** — General Overview: perspective, functions, user classes.
- **Section 3** — Specific Requirements: Functional (FRs), Non-Functional (NFRs), Interface.
- **Section 4** — From User Needs to Technical Specifications: elicitation and UML models.
- **Section 5** — Software Life Cycle and methodology.
- **Section 6** — Feasibility and considerations.
- **Section 7** — Testability and verification.

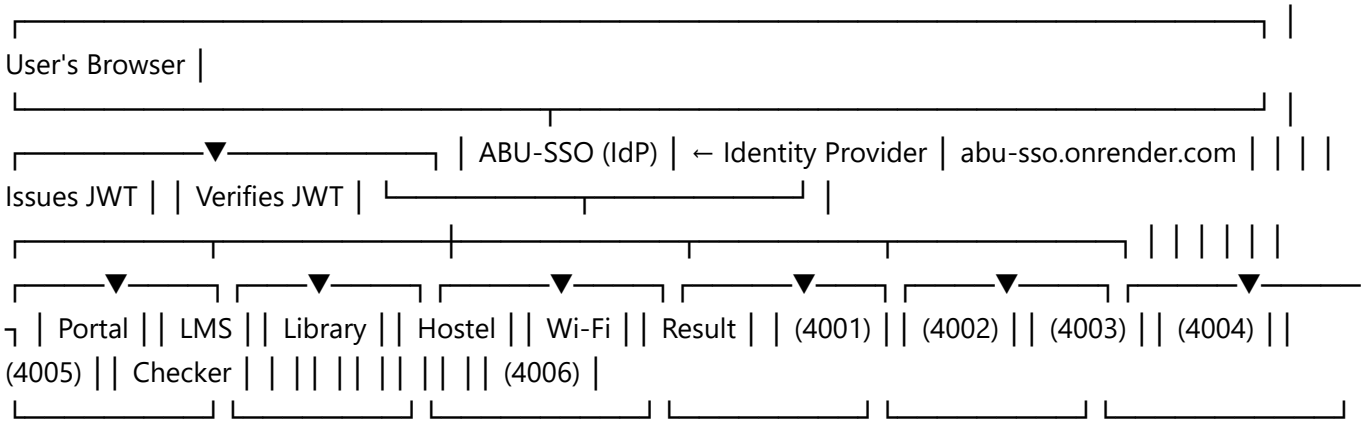
---

2. General Overview

2.1 Product Perspective

ABU-SSO is a **distributed, service-oriented system** in which a central identity provider (IdP) issues JWTs and six independent services trust those tokens. Each service is a **separate FastAPI application, deployed as a separate process**, in line with modern microservice principles.

Architectural position:



2.2 Product Functions

1. **User Account Management** — Login, logout, profile retrieval; account creation performed by administrators after the initial bootstrap.
2. **Authentication & Session Management** — Issue and validate JWT access tokens; enforce 15-minute expiry.
3. **Digital ID Card Generation** — Produce a QR-encoded identity card for each user.
4. **Campus Wallet** — Self-service top-up, payment, and transaction history.
5. **Web Dashboard** — Launchpad for connected services; shows wallet, ID card, and service cards.
6. **Admin Panel** — User management, role changes, activation/deactivation, wallet top-ups, and system statistics.
7. **Role-Based Access Control** — Distinguish students, staff, and administrators.
8. **Distributed SSO** — Six independent services that trust ABU-SSO as the identity provider.
9. **Open API for Third-Party Services** — REST endpoints documented via OpenAPI/Swagger.

2.3 User Classes and Characteristics

User Class	Description	Skill Level	Frequency of Use
Students	Undergraduate and postgraduate students using the system to access campus services, view ID card, and manage wallet. Log in with their **matriculation number**.	Basic computer literacy; familiar with web apps.	Daily
Staff	Lecturers and administrative staff using the system for teaching and library services. Log in with their **staff number**.	Basic to intermediate.	Daily
System Administrators	IT staff at the ICT Directorate. The **only** role that can create new user accounts, change roles, deactivate accounts, and top up wallets.	Advanced.	Occasional
Third-Party Services (Machine Actors)	Six distributed campus services that consume the JWT for authentication.	N/A — programmatic.	Continuous

2.4 Operating Environment

- **Server:** Linux server on Render cloud platform.
- **Database:** SQLite for development; **Neon-hosted PostgreSQL 18** in production.
- **Client:** Any modern browser (Chrome, Edge, Firefox, Safari) on desktop or mobile.
- **Network:** HTTPS in production; HTTP acceptable for local development.

2.5 Design and Implementation Constraints

- **Technology stack locked to Python 3.12 + FastAPI.**
- **JWT-based authentication** — stateless, no server-side sessions.
- **Password hashing with bcrypt** — plain-text storage prohibited.
- **Matric-number login** — no email-based authentication.
- **Bootstrap-only public registration** — after the first admin account exists, `/auth/register` returns HTTP 403.
- **Admin-only account creation** — new students/staff are created by administrators via `POST /admin/users`.
- **Source code must be managed in Git and pushed to a public GitHub repository.**
- **Automated tests must run on every push via CI.**

2.6 Assumptions and Dependencies

- Students are assigned a matric number by the university at admission time.
- Staff are assigned a staff number by HR.
- Administrators are trusted ICT Directorate staff.
- Network connectivity exists between the client browser and the server.
- Third-party services trust the JWT signature using a shared HS256 secret during the prototype; production would use RS256 public-key verification.

2.7 Apportioning of Requirements

Requirements prioritised using the MoSCoW method:

Priority	Requirements
Must have	FR-01 to FR-14, FR-16, FR-17, NFR-01 to NFR-05
Should have	FR-15, NFR-06 to NFR-09
Could have	Admin dashboard enhancements, SMS alerts
Won't have (this release)	Biometric authentication, real payment gateway

---

3. Specific Requirements

3.1 Functional Requirements (FRs)

ID	Requirement	Priority	Actor
FR-01	The system shall allow **the first-ever user** to register (bootstrap). This first user is automatically granted the `admin` role. After this, public registration shall be disabled.	Must	System
FR-02	The system shall reject registration attempts with an email or matric/staff number that already exists.	Must	System
FR-03	The system shall hash all passwords using bcrypt before storage; the plaintext password shall never be persisted.	Must	System
FR-04	The system shall allow a registered user to log in using their **matric / staff number** and password (email-based login is not supported).	Must	Student, Staff, Admin
FR-05	On successful login, the system shall return a signed JWT access token with a 15-minute expiry.	Must	System
FR-06	The system shall allow the authenticated user to retrieve their own profile (`/auth/me`) using the JWT.	Must	All authenticated users
FR-07	The system shall generate a digital ID card containing the user's name, matric number, role, and a QR code encoding `{id, matric_no, role}`.	Must	Student, Staff
FR-08	The system shall automatically create a wallet (starting balance ₦0.00) for a user on first access.	Must	System
FR-09	The system shall allow an authenticated user to top up their wallet with a positive amount.	Must	Student, Staff
FR-10	The system shall allow an authenticated user to pay from their wallet, provided the balance is sufficient.	Must	Student, Staff
FR-11	The system shall record every top-up and payment as a `credit` or `debit` transaction with a timestamp.	Must	System
FR-12	The system shall allow an authenticated user to view their transaction history (most recent first).	Must	Student, Staff
FR-13	The system shall distinguish between user roles: `student`, `staff`, and `admin`.	Must	System
FR-14	The system shall expose a health-check endpoint (`/health`) for monitoring.	Must	System, DevOps
FR-15	The system shall render a web-based login page and a dashboard for authenticated users.	Must	Student, Staff, Admin
FR-16	The system shall allow an administrator to create new user accounts via `POST /admin/users` with a specific role.	Must	Admin

ID	Requirement	Priority	Actor
FR-17	The system shall provide six independently-deployable SSO services — Student Portal, LMS, Library, Hostel, Wi-Fi, Result Checker — each of which independently verifies the ABU-SSO JWT via the IdP's `/auth/me` endpoint.	Must	System
FR-18	The system shall allow an administrator to change a user's role, activate/deactivate an account, and top up any user's wallet.	Must	Admin
FR-19	The system shall display system-wide statistics (total users, active users, wallets, total balance, transactions) to administrators.	Should	Admin

3.2 Non-Functional Requirements (NFRs)

ID	Category	Scenario	Metric
NFR-01	Performance	Login response under normal load	p95 latency ≤ 500 ms
NFR-02	Security (Confidentiality)	Passwords hashed with bcrypt	bcrypt cost ≥ 12
NFR-03	Security (Authentication)	Protected endpoints reject missing/expired tokens	HTTP 401 on invalid token
NFR-04	Security (Integrity)	JWT signed with HS256, secret ≥ 32 bytes	RFC 7518 compliant
NFR-05	Security (Registration)	Public registration disabled after bootstrap	HTTP 403 after first user
NFR-06	Reliability	`/health` returns 200 whenever running	Continuous uptime
NFR-07	Usability	New user can log in within 30 seconds of landing on the login page	Task-completion ≤ 30 s
NFR-08	Maintainability	Codebase modular: routers, services, models, schemas	Each function complexity ≤ 10
NFR-09	Testability	Automated tests cover the app	≥ 70% line coverage (achieved: 96%)
NFR-10	Portability	Runs on Windows (dev) and Linux (prod)	Docker/cloud deployable
NFR-11	Interoperability	All endpoints documented via OpenAPI 3.0	Auto-generated Swagger at `/docs`
NFR-12	Scalability	Stateless JWT auth enables horizontal scaling	Load-balancer-ready

ID	Category	Scenario	Metric
NFR-13	Availability	System deployed on free-tier cloud	99% uptime target

3.3 Interface Requirements

3.3.1 User Interfaces

- **Login page (`/login`)** — matric/staff number + password; error messages in red; success redirects to `/dashboard`.
- **Dashboard (`/dashboard`)** — welcome message, wallet balance, transaction list, QR ID card, and **six clickable service cards**.
- **Admin page (`/admin`)** — visible only to `admin` role; user list, create-user form, role changer, activate/deactivate buttons, wallet top-up form, system statistics.
- **Swagger UI (`/docs`)** — auto-generated interactive API documentation.

3.3.2 Software Interfaces

External System	Interface	Protocol	Purpose
Student Portal (mock)	REST	HTTPS/JSON	SSO login via JWT
LMS / Moodle (mock)	REST	HTTPS/JSON	SSO login via JWT
Library (mock)	REST	HTTPS/JSON	SSO login via JWT
Hostel (mock)	REST	HTTPS/JSON	SSO login via JWT
Wi-Fi (mock)	REST	HTTPS/JSON	SSO login via JWT
Result Checker (mock)	REST	HTTPS/JSON	SSO login via JWT
Database	SQLAlchemy + psycopg	SQLite / PostgreSQL	Persistent storage

3.3.3 Hardware Interfaces

No specialized hardware is required for the prototype. In a real deployment, QR codes on the digital ID card would be scanned by handheld scanners at library and hostel gateways.

3.3.4 Communication Interfaces

- All client-server communication uses HTTPS in production (TLS 1.2+).
- JWTs are transmitted in the `Authorization: Bearer <token>` HTTP header.
- JSON is the only data format for API request/response bodies (except HTML pages).

---

4. From User Needs to Technical Specifications

4.1 Requirements Elicitation

Technique	Application to ABU-SSO
Interviews	Informal interviews with 3 ABU students about frustration with multiple credentials (portal, LMS, Wi-Fi, hostel).
Ethnography / Contextual Inquiry	Observation of students in the Faculty of Engineering computer lab struggling to log in to the LMS after resetting their portal password.
Document Analysis	Review of ABU ICT Directorate's policy on the "once-only" registration principle and matric number assignment.
Prototyping	The working prototype serves as a live specification; iterative feedback refined the wallet and admin workflows.

Justification: Ethnography was prioritised because interviews revealed *what* students wanted ("one login"), but observing them revealed *why* existing systems fail (password fatigue + forgotten credentials). This is the tacit knowledge Sommerville (2016) warns is lost by interviews alone.

4.2 Use Case Model

Actors: Student, Staff, Administrator, Third-Party Service.

Primary use cases:

UC-ID	Use Case	Primary Actor
UC-01	Bootstrap first admin	First visitor
UC-02	Log In with matric number	Student, Staff, Admin
UC-03	View Profile	All authenticated
UC-04	View Digital ID Card	Student, Staff
UC-05	Top Up Wallet	Student, Staff
UC-06	Pay from Wallet	Student, Staff
UC-07	View Transaction History	Student, Staff
UC-08	Access Third-Party Service via SSO	Student, Staff
UC-09	Administer Users (create, role, activate/deactivate)	Administrator
UC-10	Health Check	Monitoring

Diagrams in `docs/diagrams/`.

4.3 Sequence Model

See `docs/diagrams/sequence_login.png` for the login sequence.

4.4 Class / Entity Model

See `docs/diagrams/class_model.png`.

4.5 Functional Decomposition

ABU-SSO ├── Authentication Subsystem │ ├── Bootstrap registration (FR-01) │ ├── Login with matric (FR-04, FR-05) │ └── Identity resolution (FR-06) ├── Admin Subsystem │ ├── Create users (FR-16) │ ├── Manage users (FR-18) │ └── Statistics (FR-19) ├── Identity Subsystem │ └── Digital ID Card + QR (FR-07) ├── Wallet Subsystem │ ├── Auto-create (FR-08) │ ├── Top up (FR-09, FR-11) │ ├── Pay (FR-10, FR-11) │ └── History (FR-12) ├── Presentation Subsystem │ ├── Login page │ ├── Dashboard (FR-15) │ └── Admin panel └── Distributed SSO Subsystem ├── Portal, LMS, Library └── Hostel, Wi-Fi, Result Checker (FR-17)

4.6 Evaluation Metrics and KPIs

Goal	KPI	Target
Fast login	p95 `/auth/login` latency	≤ 500 ms
Secure credentials	% passwords hashed with bcrypt	100%
Reliable auth	Unauthorised requests rejected	100%
Registration security	Post-bootstrap `/auth/register` rejected	100% (HTTP 403)
Test coverage	pytest line coverage	≥ 70% (achieved: 96%)
CI health	GitHub Actions pass rate	100%

---

5. Software Life Cycle Considerations

5.1 Development Methodology

Iterative and incremental **Agile**, delivered in **12 vertical slices** (see Technical Report §2.1). Justification: the problem statement involves real users with evolving needs.

5.2 Version Control

Git with a public GitHub repository; trunk-based development; Conventional Commits.

5.3 Continuous Integration

GitHub Actions runs the full test suite on every push to `main` (see `.github/workflows/ci.yml`).

5.4 Test Planning

See Section 7.

---

6. Feasibility and Considerations

6.1 Technical Feasibility

Aspect	Assessment
Skills	Python + FastAPI is well-documented; prototype completed with free tutorials and library docs.
Maturity	FastAPI, SQLAlchemy, bcrypt, PyJWT are industry-standard.
Hardware	Any modern laptop (4 GB RAM); production on Render free tier.
Integration	JWT-based SSO is a widely adopted pattern.

Verdict: Technically feasible.

6.2 Economic Feasibility

Item	Cost
Developer time	₦0 (academic)
Tooling	₦0 (open-source)
Hosting (Render + Neon free tiers)	₦0
Total operational	**₦0**

Verdict: Economically feasible.

6.3 Risk Assessment

See `docs/RISK_AND_MAINTENANCE.md`.

6.4 Testability Considerations

Three-layer test pyramid + E2E layer. Tests use a **shared in-memory SQLite** with `StaticPool` to guarantee a single DB across all sessions.

---

7. Testability

7.1 Testing Strategy

Four-layer pyramid: unit → service → integration → system (E2E).

7.2 Test Coverage Goals

Level	Target	Achieved
Unit	100% of `security.py`	✅
Service	100% of services	✅
Integration	All HTTP routes	✅
System (E2E)	5 critical user journeys	✅

Level	Target	Achieved
Total tests	—	**84 passing**

7.3 Test Environment

- Temporary SQLite per test session (via `StaticPool`).
- Fixtures in `tests/conftest.py` (`client`, `db`, `live_server`, `registered_user`).
- Auto-cleanup between tests.

7.4 Verification Against Requirements

See `docs/RTM.md`.

Appendix A — Assumptions

See `docs/TECHNICAL_REPORT.md` § 11 for the complete list.

Appendix B — Future Work

- OIDC / OpenID Connect with RS256 asymmetric keys.
- Audit log table recording all admin actions.
- Rate limiting via `slowapi`.
- Refresh tokens for better UX.
- Email verification + OTP password reset.
- Real payment gateway integration.
- Native mobile app reusing the same API.

End of Software Requirements Specification (SRS).